Ant Colony Optimization on a Fractal Landscape

Short presentation of the ENSTA project

Project scope

- Goal: optimize the simulation code, not redesign the model.
- Main steps: sequential baseline, SoA restructuring, OpenMP, MPI.
- Key question: which optimization really improves performance?

Environment
513 x 513 grid

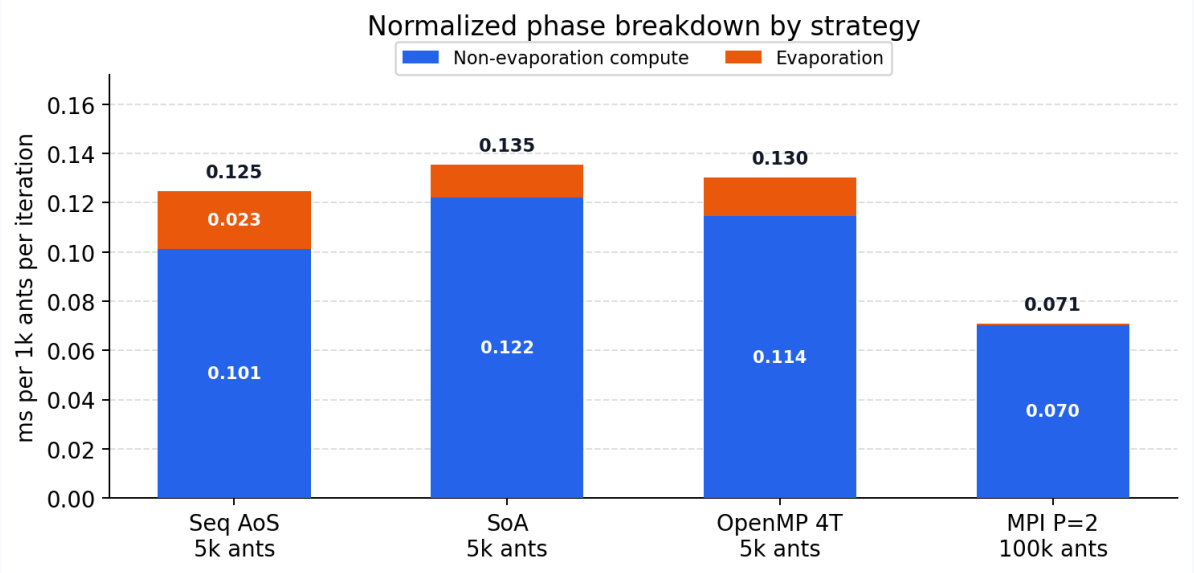
Reference load
5,000 ants

MPI test case
100,000 ants

1. Problem and baseline

What the original simulation does before optimization

- 2D fractal terrain with one nest, one food source, and two pheromone fields.
- Each ant moves step by step and updates pheromone information during the simulation.
- The first benchmark isolates the main phases: ant movement, evaporation, update, and display.



Seq total
0.623 ms/iter

Dominant phase
Ant movement

Display
Large with VSync

Takeaway: once display is excluded, ant movement is the main optimization target.

2. SoA and OpenMP

Two first optimization attempts

AoS → SoA restructuring

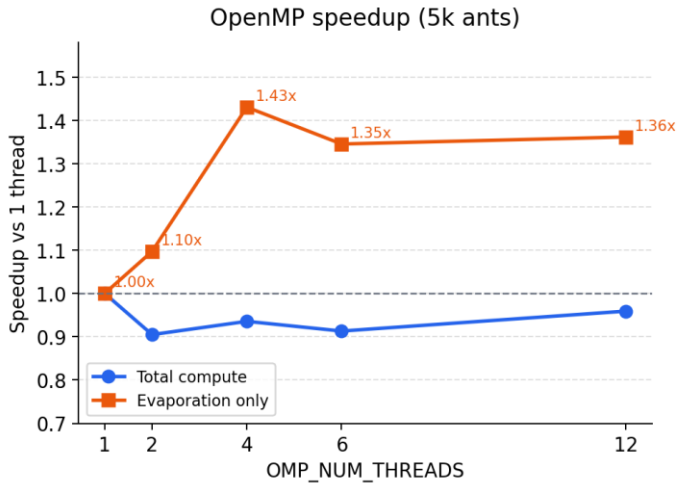
- Objects replaced by arrays: x, y, state, seed.
- Same simulation logic as the baseline.
- Result: slower, because the kernel is still irregular and ant-by-ant.

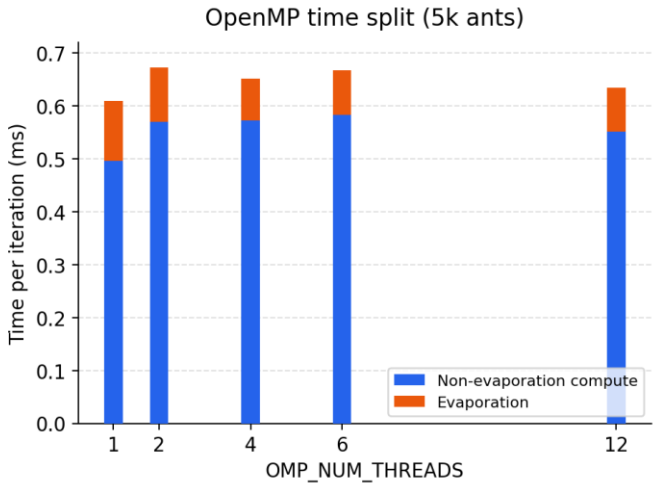
Normalized result

0.92x vs Seq

OpenMP on evaporation only

Why it helps only a little: evaporation scales, but it is only a small fraction of total





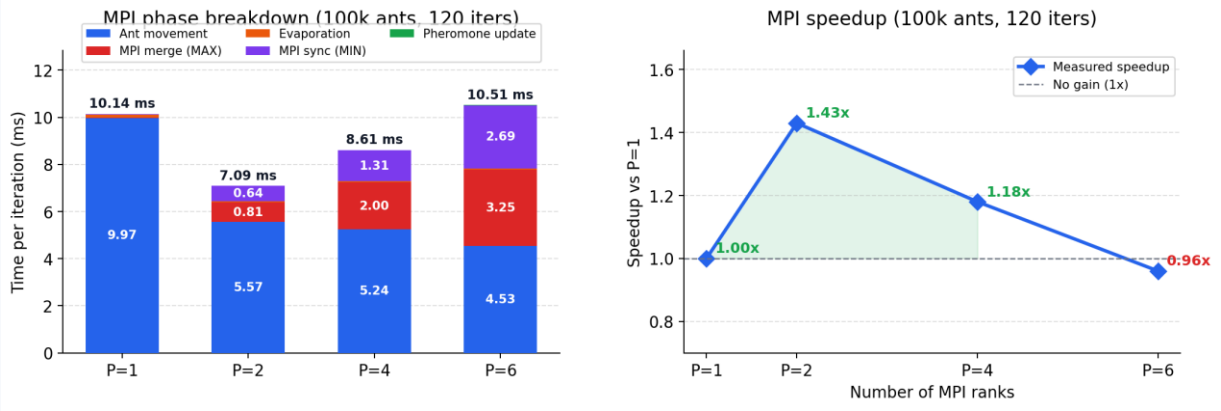
Best evaporation speedup

1.43x

3. MPI strategy

Approach 1 from the project statement

- Each rank keeps the full map, but only a subset of ants.
- Two collective communications per iteration: MAX merge after ant updates, then MIN sync after stripe evaporation.
- This strategy can win only if the ant workload is large enough to amortize communication.



Best point
P=2

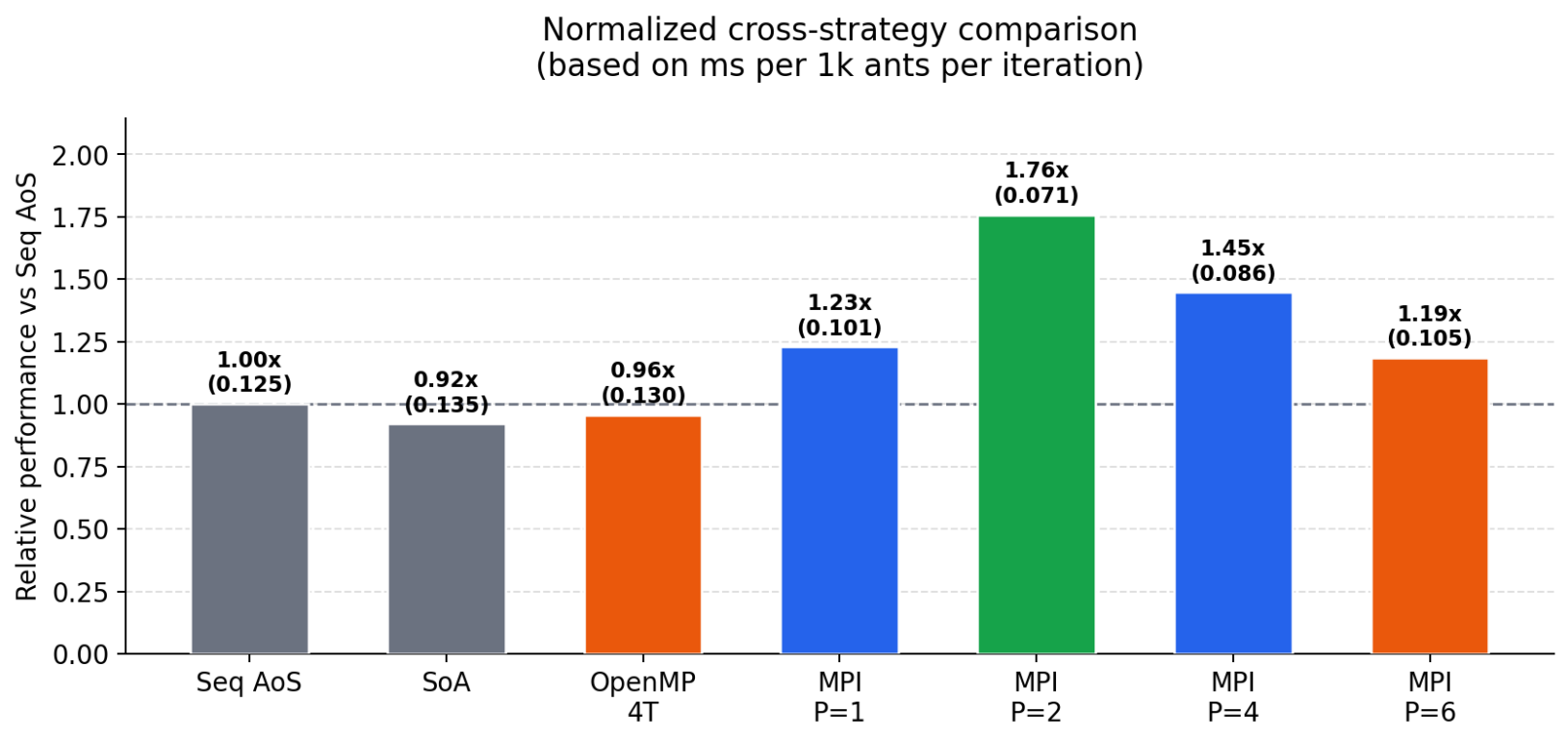
Best speedup
1.43x

Why P=6 drops
Communication overhead

The MPI result is positive only for large loads: with small workloads, communication dominates.

4. Final comparison

Which strategy gives the best practical result?



Main conclusions

- SoA alone does not help this irregular kernel.
- OpenMP is useful but too local to change the full result.
- MPI gives the best normalized result, but only for a large enough workload.

Best normalized result
1.76x vs Seq

5. One-minute conclusion

What we learned from the project

- Measure first: the true bottleneck was not where we initially expected.
- Data-layout changes are not enough when control flow and memory access stay irregular.
- A safe OpenMP step improved only one phase, which matched Amdahl's law.
- MPI can outperform the sequential code, but only when the problem size justifies the communication cost.

Optimization must follow the structure of the workload.

The best result came from the strategy that matched the dominant cost at scale.

Thank you